

JavaScript Array.prototype

Learning Notes — From Arrays to the Prototype Chain

Goal: understand what Array.prototype is, why array methods work, and how this connects to JavaScript OOP, classes, objects, and inheritance.

1. What is an Array?

An array is an object used to store an ordered collection of values.

```
const numbers = [10, 20, 30];
```

The variable **numbers** refers to an Array object. The array can use methods such as push(), pop(), map(), filter(), and many others.

2. What is Array.prototype?

Array.prototype is an object containing methods and properties that are shared by Array instances. When you create an array, JavaScript connects that array to Array.prototype through its prototype chain.

Think of it as a shared toolbox for arrays:

```
numbers → Array.prototype → array methods
```

Examples of methods found on Array.prototype include push(), pop(), map(), filter(), forEach(), find(), and includes().

3. Prototype Lookup — The Most Important Idea

Suppose you write:

```
const numbers = [1, 2, 3];  
numbers.push(4);
```

JavaScript needs to find the property named **push**. It first checks the array object itself. If it does not find push there, JavaScript follows the object's prototype link and checks Array.prototype.

Lookup flow:

```
numbers  
↓  
Does numbers have push?  
↓ No  
Array.prototype  
↓  
push found!  
↓  
Execute push()
```

4. Verify It Yourself

You can inspect the prototype of an array using `Object.getPrototypeOf()`.

```
const arr = [1, 2, 3];

console.log(Object.getPrototypeOf(arr));

console.log(Object.getPrototypeOf(arr) === Array.prototype);
```

The final statement evaluates to **true**. This demonstrates that the prototype of a normal array is `Array.prototype`.

5. `__proto__` vs prototype

These two terms are related but are not the same thing.

Term	Meaning
prototype	A property commonly found on constructor functions/classes. It is the object used as the prototype for instances.
<code>__proto__</code>	A legacy accessor that exposes an object's prototype. Prefer <code>Object.getPrototypeOf()</code> in modern code.
<code>Object.getPrototypeOf(obj)</code>	The modern, explicit way to retrieve an object's prototype.

6. Array Methods Come From the Prototype

For example:

```
const numbers = [10, 20, 30];

numbers.push(40);
numbers.pop();
numbers.includes(20);
numbers.map(x => x * 2);
numbers.filter(x => x > 10);
```

You do not manually define these methods for every array. They are available through the shared `Array.prototype`.

7. A Simple Mental Model

Imagine `Array.prototype` as a shared toolbox. Every normal array can reach that toolbox through its prototype link.

```
Array.prototype
  ■■■ push()
  ■■■ pop()
  ■■■ map()
```

```
filter()
find()
includes()
forEach()
```

8. Creating Your Own Prototype Method

JavaScript technically allows you to add a method to `Array.prototype`:

```
Array.prototype.sayHello = function () {
  console.log("Hello");
};

const numbers = [1, 2, 3];
numbers.sayHello();
```

This works because numbers can find `sayHello` through `Array.prototype`. However, modifying built-in prototypes is generally discouraged in application code because it can create naming conflicts and unexpected behavior.

9. Connection to JavaScript Classes

This is where prototypes connect directly to OOP.

```
class BankAccount {
  constructor(balance) {
    this.balance = balance;
  }

  deposit(amount) {
    this.balance += amount;
  }
}
```

When you create an object:

```
const account = new BankAccount(1000);
```

the object can access `deposit()` through `BankAccount.prototype`. The same basic prototype idea explains why arrays can access methods through `Array.prototype`.

10. `Array.prototype` vs `BankAccount.prototype`

Array	Your Class
<code>const arr = [];</code>	<code>const account = new BankAccount(1000);</code>
<code>Array.prototype</code>	<code>BankAccount.prototype</code>
<code>push()</code> , <code>pop()</code> , <code>map()</code> , <code>filter()</code>	<code>deposit()</code> , <code>withdraw()</code> , etc.

11. Prototype Chain

If a property is not found directly on an object, JavaScript can continue looking through its prototype chain.

```
arr
↓
Array.prototype
↓
Object.prototype
↓
null
```

This is why arrays can use both array-specific methods and methods inherited from `Object.prototype`.

12. Interview-Style Questions

1. What is `Array.prototype`?
2. Why can every normal array call `push()`?
3. What does `Object.getPrototypeOf([1, 2, 3])` return?
4. What is the difference between `prototype` and `__proto__`?
5. How does prototype lookup work when a property is missing?
6. How does `BankAccount.prototype` relate to `Array.prototype`?

13. Practice

Try these in your JavaScript console before checking your understanding:

```
const arr = [10, 20, 30];

// A
console.log(Object.getPrototypeOf(arr) === Array.prototype);

// B
console.log(arr.hasOwnProperty("push"));

// C
console.log(Array.prototype.hasOwnProperty("push"));
```

Think carefully about B and C. The goal is to understand where push is actually stored and how the array reaches it.

14. Key Takeaways

- Arrays are objects.
- Array.prototype contains shared array methods.
- An array instance is linked to Array.prototype.
- JavaScript searches the prototype chain when a property is not found directly on the object.
- Object.getPrototypeOf(obj) is the modern way to inspect an object's prototype.
- Classes also use prototypes: instance methods are associated with ClassName.prototype.
- Understanding prototypes makes JavaScript OOP and inheritance much easier to understand.